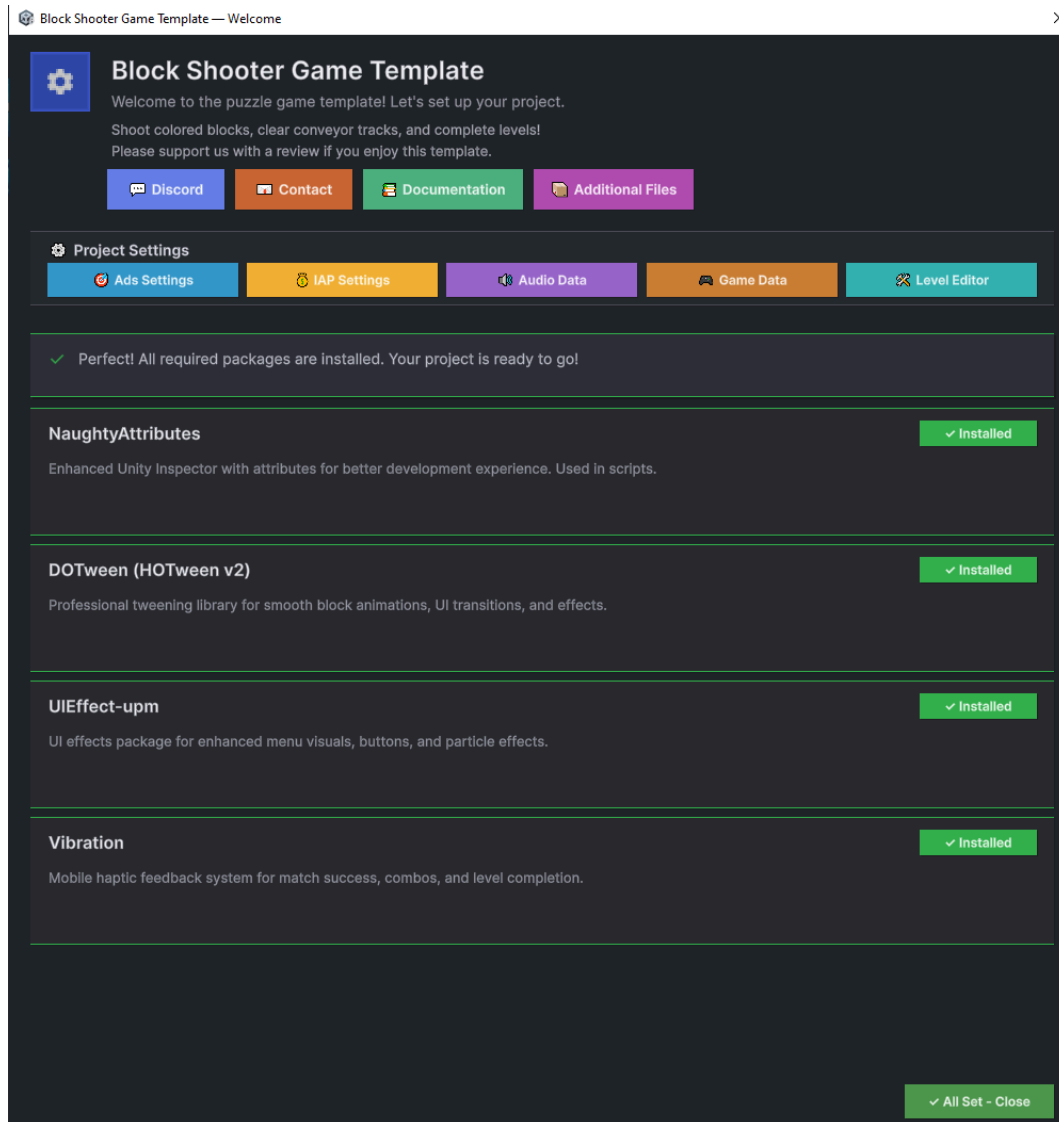




Block Shooter | Documentation

▼ IMPORTANT: Required Package Setup



This template relies on standard Unity components and a few key packages. You can verify and install dependencies using the built-in **Welcome Guide** ([Tools > Welcome Guide](#)), which automatically detects missing packages and helps configure your project.

Required Packages

Package / Dependency	Purpose	Source / Verification
DOTween (HOTween v2)	Handles all smooth block animations, UI scaling punches, and projectile path arcing.	[Demigiant
NaughtyAttributes	Enhances Unity inspector layout with buttons, foldouts, and conditional formatting.	[dbrizov
UIEffect-upm	Provides premium visual effects for UI images, including glows, dissolves, and shiny highlights.	[mob-sakai
Vibration	Enforces haptic feedback triggers specifically on box destructions and game actions.	[Elringus

▼ Core Game Mechanics

1) Shooter Grid Deck & Slots

- **Shooter Blocks:** Placed in the deck area. Each block has a color type and a specific ammo count.
- **Slot System:** Players tap a block in the deck to move it to one of the active slots (default is 4 slots, expandable to 5 slots with boosters/revives).
- **SlotSystem Controller (`SlotSystem.cs`):** Manages slot references, projectile spawning delays, and clears slots when shooter blocks deplete.

2) Conveyor & Track Movement System

- **Spline-Based Tracks:** Conveyor blocks travel along path splines generated via the Level Editor.
- **ConveyorController (`ConveyorController.cs`):**
 - Controls progression calculations (mapping percentage values between `0.0f` and `1.0f`) to move block groups smoothly.
 - Handles **Branch Paths** connection merges, allowing sub-conveyors to feed blocks seamlessly into the main belt.
 - Adjusts speed dynamically (e.g. during end-game acceleration or x2 speed multiplier).
- **Conveyor Blocks (`ConveyorBlock3D`):** Individual target boxes of various color configurations carrying active mesh elements.

3) Firing & Homing Projectile System

- **FireRange Scan & Customization:** Active slots continuously scan for incoming target boxes using visual area sweeps (`FireRange.cs`). The `FireRange` prefab is located in the

`Prefabs` directory and used within the levels. Developers can increase the horizontal scale/width of this prefab in Unity to expand the active targeting and shooting range of the slots.

- **Homing Projectiles (`Projectile.cs`):** Projectiles automatically seek out their targeted conveyor block using frame-rate independent transform homing interpolation. This guarantees hits regardless of device frames-per-second.
- **ProjectilePool (`ProjectilePool.cs`):** Implements an object pooling pattern to recycle bullet objects. This drastically reduces Garbage Collection allocation overhead, ensuring smooth gameplay on low-end mobile devices.
- **Haptic Feedback:** Triggers a pop vibration (`Vibration.VibratePop()`) on target box impact and destruction.

4) Booster System & Booster UI Interaction

Location: Core logic is handled by `BoosterManager.cs` while individual button slots are managed by `BoosterSlotUI.cs`.

Players purchase and trigger boosters to escape failing conditions. The three booster types function as follows:

- **Extra Slot (Instant Activation):**
 - **Function:** Adds an extra active slot to the player's deck slot row (spawning a new slot UI indicator via `SlotSystem.Instance.AddExtraSlot()`).
 - **Usage:** Instantly increases slot capacity from the default of `4` up to a maximum limit of `5`. This gives players more temporary storage to unload blocks and avoid slot grid deadlocks.
 - **How to Trigger:** Tapping the Extra Slot button activates it immediately without target selection.
- **Super Shooter (Target Selection Mode):**
 - **Function:** Upgrades standard shooter block ammo to explosive, high-impact rounds. When these projectiles strike a target conveyor block, they detonate, clearing the target as well as adjacent, matching-color blocks.
 - **Global Target Overrides:** Once a shooter block on a slot is upgraded with Super Shooter, its firing mechanism **bypasses the standard FireRange constraints**. It will automatically target and shoot matching color blocks anywhere along the conveyor, regardless of how far away they are, and blow up matching targets.
 - **How to Trigger:** Tapping the booster button sets `IsAwaitingSuperShooterTarget` to true and triggers a breathing glow animation (`glowActive`) on the button. The player must then tap any loaded shooter block on the 7×6 grid deck to apply the upgrade.
- **Move Shooter (Grid Relocation Mode):**

- **Function:** Allows players to reorganize their shooter grid mid-game by moving blocks out of blocked or strategically poor cells.
- **Slot Relocation Utility:** Can be used to bypass grid blocking constraints. It enables players to select shooter blocks trapped in the middle of the grid (which normally cannot be clicked or sent to slots because other blocks block them) and send them directly to active slots.
- **Limitations:** This booster **cannot** be used to relocate or move **Mystery blocks** or **Frozen blocks**.
- **How to Trigger:** Tapping the button sets `IsAwaitingMoveShooterTarget` to true and starts the button glow. The player selects a shooter block from the deck, and then taps either an empty grid cell to relocate it, or an active slot to send it there.
- **Booster Inventory & Purchase Loop:**
 - When a player has `0` quantity of a booster, the slot displays a "+" icon (managed by `plusBuyGroup`).
 - Tapping this button pops up the booster purchase window (`buyPanel`), prompting the user to buy more boosters using coins. Prices and level locks are dynamically loaded from `GameplaySettingsConfig.asset`.

5) Core Game Loop Managers

- **GameManager (`GameManager.cs`):** Houses global state machine checks (`Normal` , `Paused` , `Frozen` , `Win` , `Fail`). Listens to conveyor box destructions to verify level clearing conditions.
- **LevelManager (`LevelManager.cs`):** Resolves scene transitions. Uses the sequence list inside `LevelSequenceConfig` to load the appropriate level prefab dynamically on scene start.

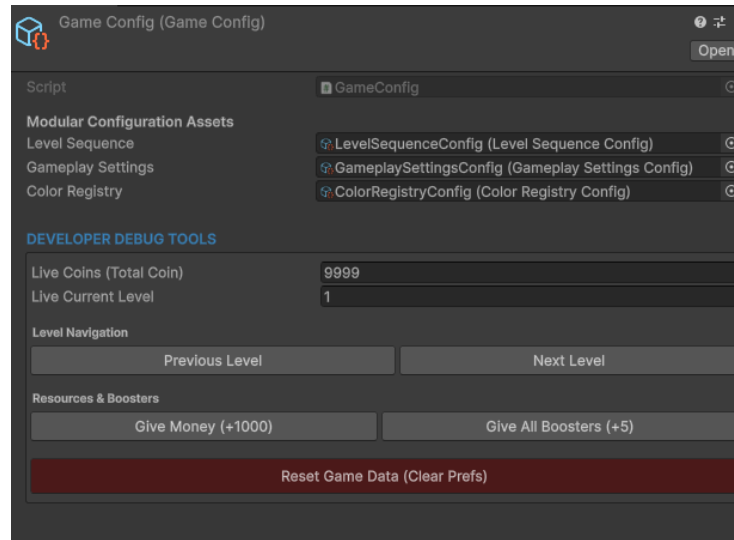
6) Win/Lose Conditions

- **Win:** Destroy all blocks travelling along the conveyor spline.
- **Lose:**
 - Active slots fill up entirely (no empty slots left) and no matches/shots can be executed.
 - The shooter deck runs out of ammo or blocks while conveyor blocks are still moving.

▼ Configuration & Customization Assets

The game logic is configured via ScriptableObjects. You can find these assets under `Assets/Project Files/Data/`.

1. GameConfig (`GameConfig.asset`)



The central entry point referencing all modular configuration assets:

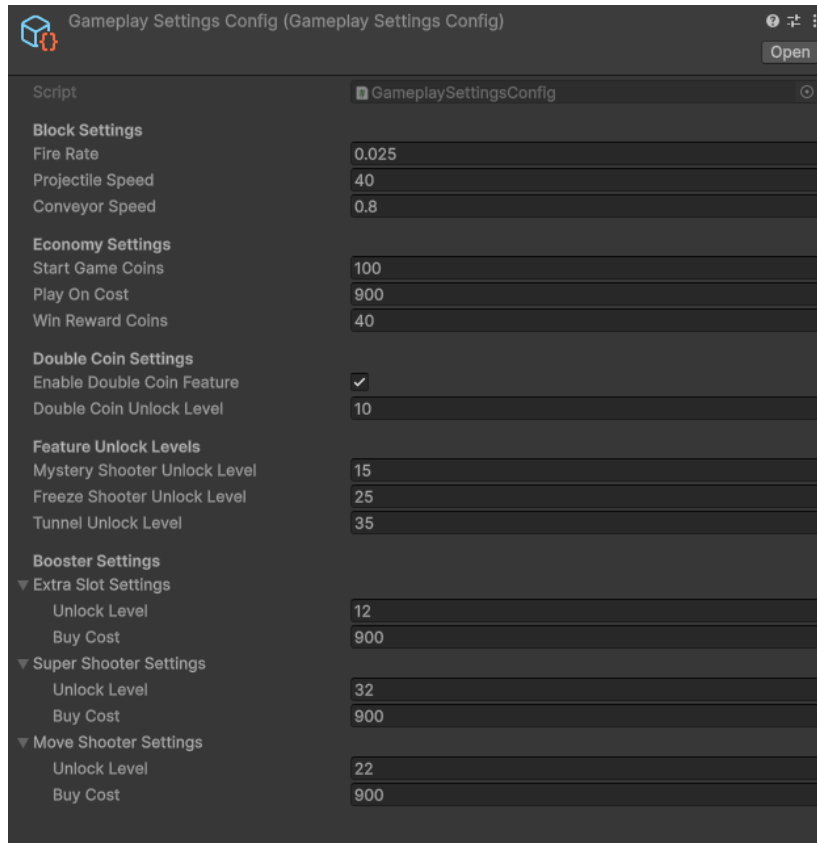
- **Level Sequence Config:** Configures the sequence of level prefabs.
- **Gameplay Settings:** Direct link to gameplay balance and pricing.
- **Color Registry:** Direct link to color material mapping.

Developer Testing & Progression Tools (Play Mode Only)

When playing in the Unity Editor, the `GameConfig` inspector provides buttons for rapid QA testing:

- **Previous Level / Next Level:** Instantly load and navigate levels.
- **Give Money:** Instantly adds `+1000` coins to the player's total.
- **Give All Boosters:** Adds `+10` of each booster type to the player inventory.
- **Reset Game Data:** Wipes all saved PlayerPrefs and JSON progress, resetting back to level 1.

2. GameplaySettingsConfig (`GameplaySettingsConfig.asset`)



Manages economy, booster costs, and gameplay behaviors:

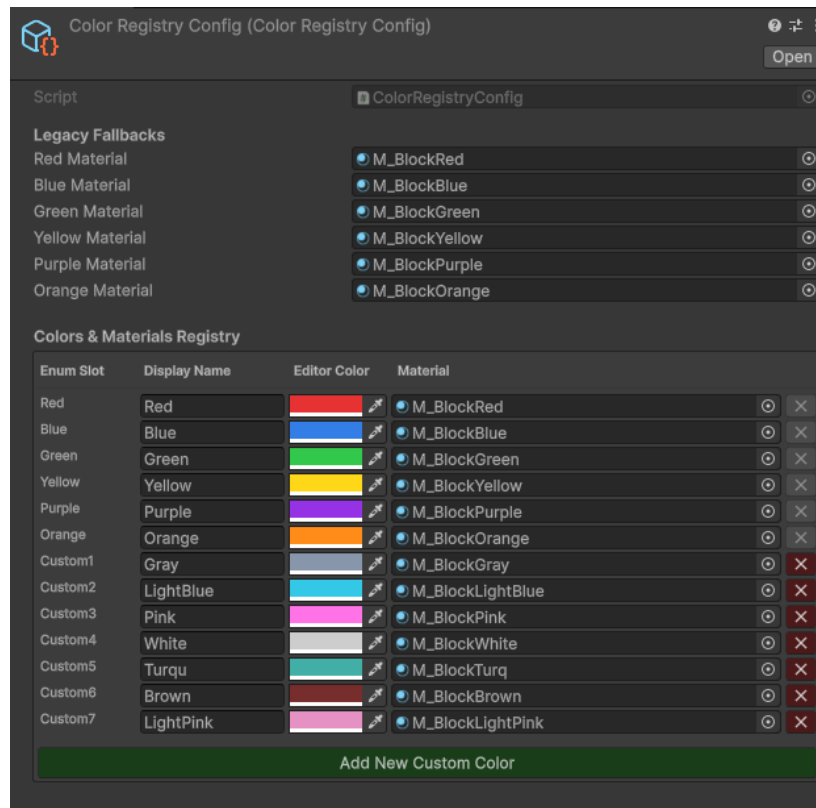
Field	Description	Default Value
fireRate	Time interval between automatic shots.	0.15 seconds
projectileSpeed	Speed of launched projectiles.	12.0
conveyorSpeed	Default speed of blocks on the track.	1.5
startGameCoins	Starting coins for new players.	100
playOnCost	Coin cost to continue/revive on failure.	100
winRewardCoins	Base coin reward given on level success.	50
enableDoubleCoinFeature	Enables a rewarded ad button to double win coins.	True
doubleCoinUnlockLevel	Level at which the double coin feature unlocks.	10
mysteryShooterUnlockLevel	Level at which Mystery Shooter blocks unlock.	4
freezeShooterUnlockLevel	Level at which Frozen Shooter blocks unlock.	3
tunnelUnlockLevel	Level at which splines with spawning tunnels unlock.	2

Booster Settings:

- **ExtraSlot:** Unlocks at Level 1 | Buy cost: 150 coins.
- **SuperShooter:** Unlocks at Level 5 | Buy cost: 250 coins.

- **MoveShooter:** Unlocks at Level 2 | Buy cost: 200 coins.

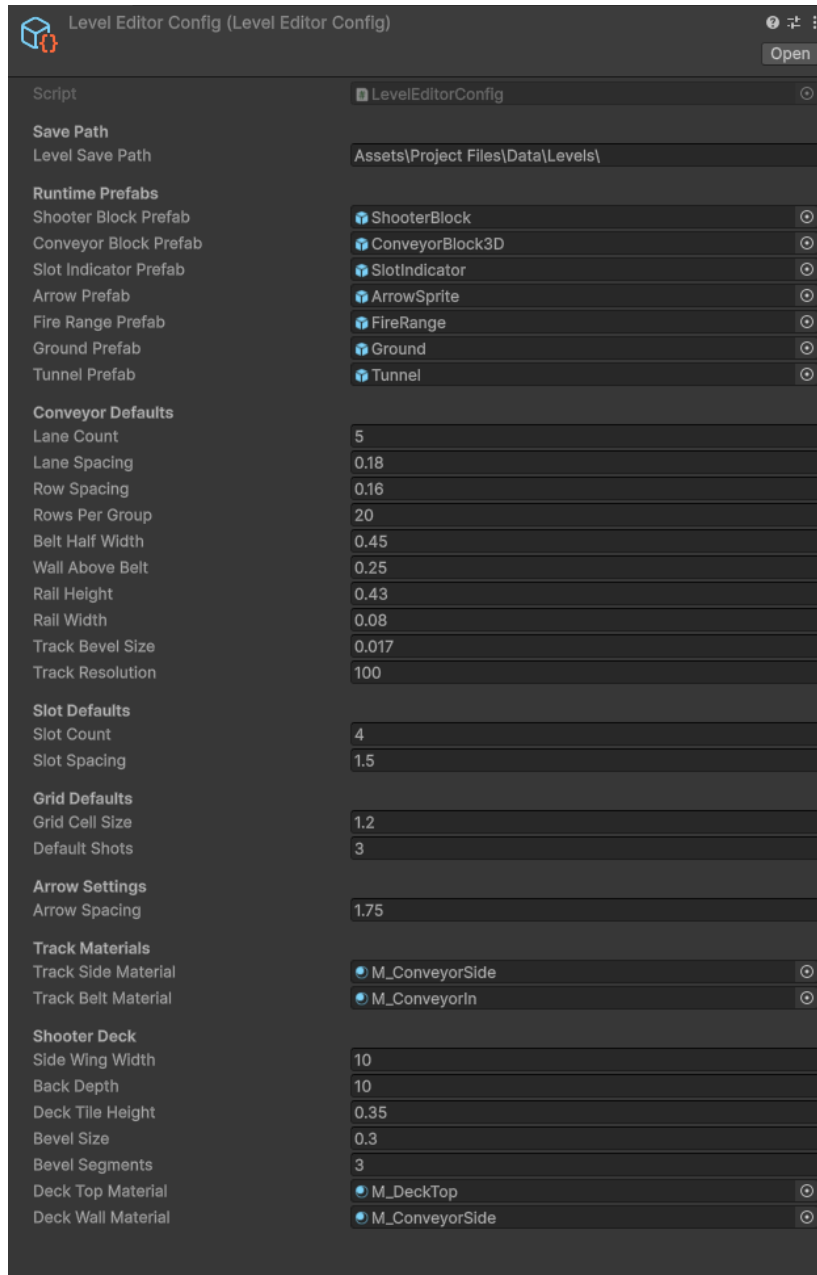
3. ColorRegistryConfig (**ColorRegistryConfig.asset**)



Binds **BlockColorType** enums to visual representations.

- Maps enum values (Red, Blue, Green, Yellow, Orange, Purple, Pink, Cyan, etc.) to specific Unity **Color** values and target **3D Materials**.
- Easily expand the palette by adding new definitions to the list.

4. LevelEditorConfig (**LevelEditorConfig.asset**)



Contains default settings, mesh generation properties, and prefab references for the Level Editor utility:

- **Save Path:** Target directory for generated level prefabs (default: `Assets/Project Files/Data/Levels/`).
- **Runtime Prefabs (Visual Customization Source):** References the base game objects spawned dynamically during gameplay and level generation. Developers can easily customize the 3D models, materials, scaling, or components of these objects by editing the corresponding prefabs located inside the `Assets/Project Files/Game/Prefabs/` folder:

- **ShooterBlock** (`ShooterBlock.prefab`): The playable shooter unit that holds ammo and colors.
- **ConveyorBlock3D** (`ConveyorBlock3D.prefab`): The target blocks moving along the conveyor spline.
- **SlotIndicator** (`SlotIndicator.prefab`): Outlines representing active slots.
- **ArrowSprite** (`ArrowSprite.prefab`): Path-direction scrolling indicators.
- **Ground** (`Ground.prefab`): The base floor/level ground geometry.
- **Tunnel** (`Tunnel.prefab`): The tunnel spawner entry/exit meshes.
- **Conveyor Defaults (Conveyor Path Mesh Settings):**
 - `laneCount` : Number of track lanes (configured to `5` by default).
 - `laneSpacing` & `rowSpacing` : Spacing parameters between lanes and rows.
 - `rowsPerGroup` : Maximum row entries per spline section.
 - `beltHalfWidth` : Half-width dimension of the conveyor belt.
 - `wallAboveBelt` & `railHeight` : Outer barrier rail heights and drop lengths.
 - `railWidth` & `trackBevelSize` : Barrier rail width and outer chamfer values.
 - `trackResolution` : Spline resolution steps (higher value means smoother curves).
- **Slot Defaults (Slot Settings):**
 - `slotCount` : Number of standard active slots.
 - `slotSpacing` : World space offset distance between slots.
- **Grid Defaults (Grid Cell Settings):**
 - `gridCellSize` : Size of each grid node in the shooter deck.
 - `defaultShots` : Starting shot capacity for new shooter blocks.
- **Arrow Settings (Directional Spline Indicators):**
 - `arrowSpacing` (default: `1.75`) : Controls the world-space distance/interval between the direction-indicator arrows generated along the conveyor track. Higher values place the arrows further apart (sparser), while lower values make them closer together (denser).
 - **Visual Customization:** Developers can easily change the arrow mesh, model, color, or sprite by modifying the `Arrow` prefab located directly in the project's `Prefabs` folder.
- **Track Materials (Conveyor Materials):**
 - `trackSideMaterial` & `trackBeltMaterial` : Renders the side structures and scrolling inner belts.

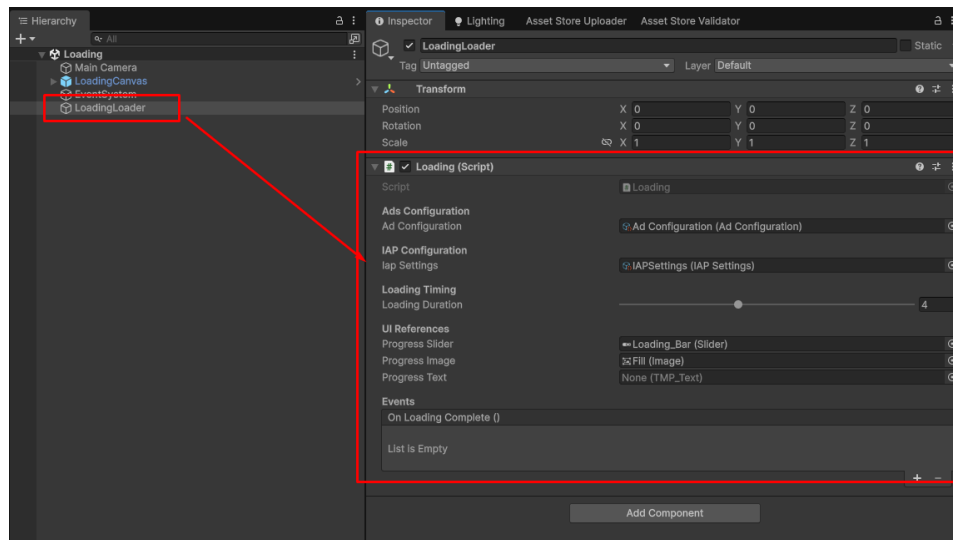
- **Shooter Deck (Shooter Deck Settings):**

- `sideWingWidth` & `backDepth` : Side extensions and rear platform thickness.
- `deckTileHeight` & `bevelSize` & `bevelSegments` : Bevel rounding/segments and height drop values for rendering the 3D grid platform.
- `deckTopMaterial` & `deckWallMaterial` : Platform base and wall surface materials.

▼ UI Canvas & Navigation Scenes

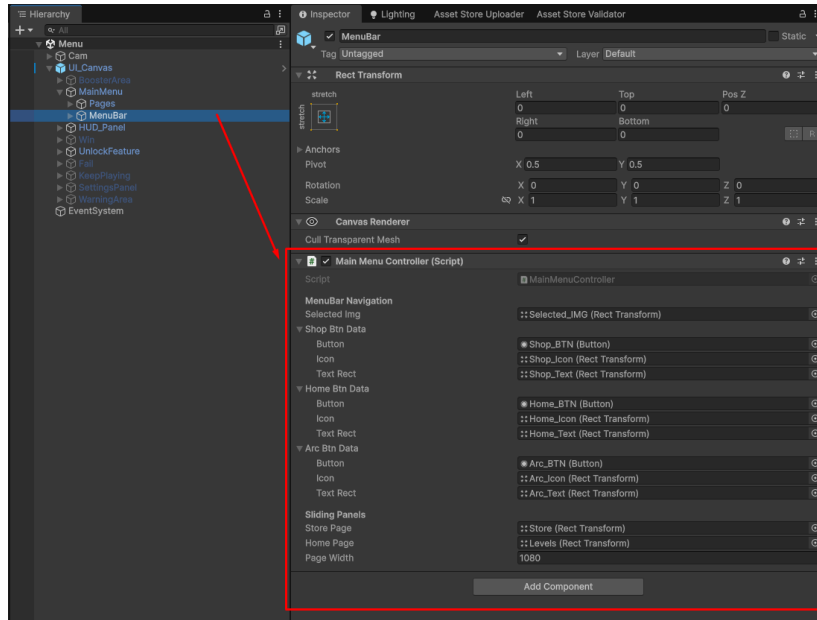
The UI is structured around standard Unity Canvas architecture and split into three core scenes:

1. Loading Scene (`Loading.unity`)



- **Role:** Initializes core configurations and saved data states on launch.
- **Flow:** Displays a splash screen, triggers the Ads settings with a GDPR consent request if enabled, pre-loads app data, and transitions automatically to the Menu scene after `loadingDuration` seconds.

2. Menu Scene (`Menu.unity`)

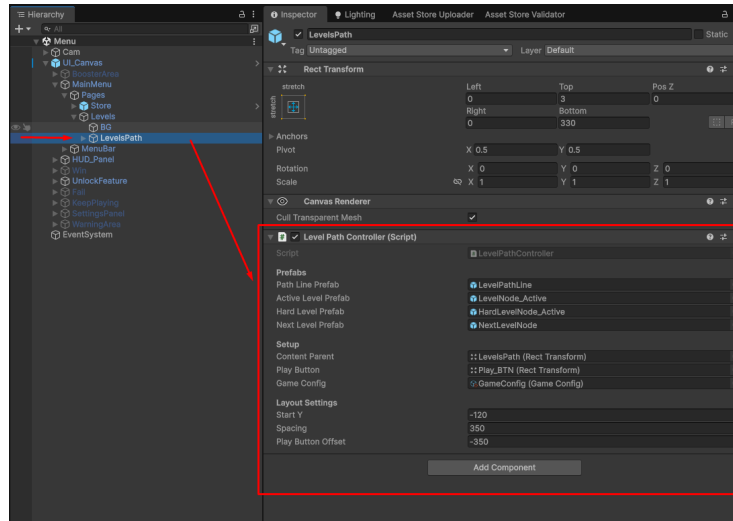


MainMenuController (Navigation & Tab Panels)

Location: `MainMenuController.cs` attached to the Canvas in the Menu scene.

- **MenuBar Navigation:** Handles switching tabs: Shop, Home, and Achievement (placeholder showing a "Coming soon!" warning popup).
- **Selection Indicator:** Moves the `selectedImg` indicator horizontally to snap below the active button and animates the active tab's icon scale (up to `1.2x`) and text offset using DOTween.
- **Page Sliding Animation:** Controls page transitions by sliding pages horizontally based on calculated screen/canvas width (`pageWidth`):
 - Shop selected: Store page (`storePage`) slides in to coordinate `0` from the left, while Home page (`homePage`) slides out to the right.
 - Home selected: Home page slides back to coordinate `0` from the right, while Store page slides out to the left and is deactivated.

LevelPathController (Level Roadmap)

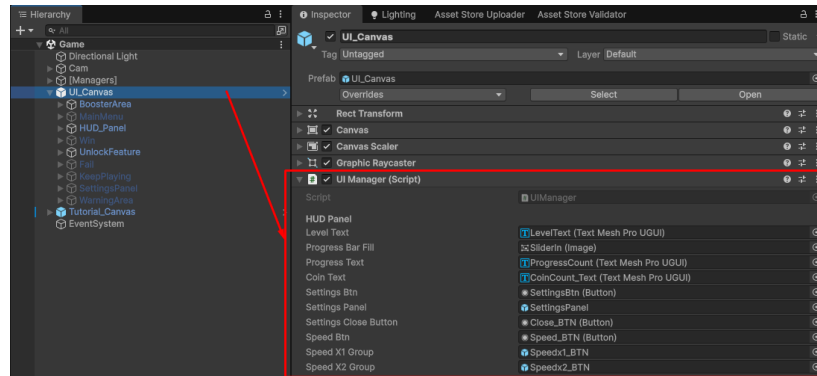


Location: `LevelPathController.cs` attached to the `Level_Path_Holder` or UI container.

- **Roadmap Assembly:** Clears existing node children and draws a vertical connect line using `pathLinePrefab` (`Path_Line`).
- **Level Progression Nodes:** Queries the active progression index via `SaveManager.CurrentLevel` and instantiates three sequential node prefabs from bottom to top:
 1. **Active Level Node:** The current playable level (uses `activeLevelPrefab` for normal stages, or `hardLevelPrefab` for hard stages).
 2. **Next Level Node:** The upcoming locked stage (uses `nextLevelPrefab`).
 3. **Two-Steps Upcoming Node:** The subsequent locked stage (uses `nextLevelPrefab`).
- **Node Prefab Types:**
 - `levelnode_active` : Standard active level circular UI node.
 - `hardlevelnode_active` : Visual variation of the active level indicating a high-difficulty layout.
 - `nextlevelnode` : Locked state representation showing upcoming stages.
- **Layout Metrics:**
 - `startY` (-120f): Vertical anchor position for the first node.
 - `spacing` (240f): Vertical gap offset between adjacent stage nodes.
 - `playButton` : Positioned dynamically directly below the active level node (`startY + playButtonOffset`). Clicking it triggers the `StartGame()` routine, which loads the Game scene.
 - **Editor Preview:** Right-click the inspector component and select **Build Level Path Preview** to visually construct and verify the roadmap layout without entering Play Mode.

3. Game Scene (`Game.unity`)

UIManager (Main UI Hub)



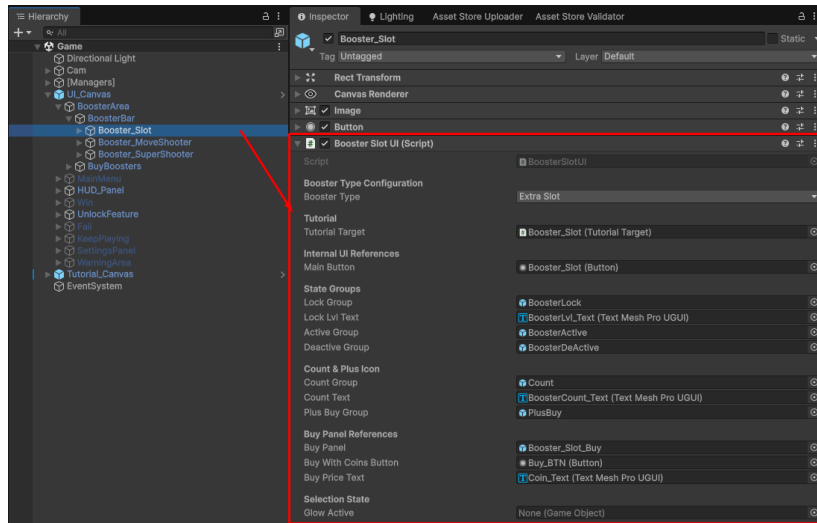
Location: `UIManager.cs` attached to the UI Canvas in the Game scene.

Manages all game overlays, text updates, button events, and animation tweens.

- **Inspector References & Groups:**

- **HUD Panel:** `levelText` (TMPPro level index), `progressBarFill` (fills based on destroyed conveyor block counts), `progressText` (TMPPro displaying percentage progress), `coinText` (tracks and animates current coins), and buttons/panels for `settingsBtn`, `settingsPanel`, `settingsCloseButton`.
- **Game Speed:** `speedBtn` which toggles between x1 speed (`speedX1Group`) and x2 speed (`speedX2Group`) by scaling and adjusting `ConveyorController.Instance.SetSpeedMultiplier`.
- **Win Panel:** `winPanel` container, `winCoinText` (displays rewards), `winNextButton`, and the rewarded video `winDoubleCoinButton` (unlocked at custom levels, doubles rewards using canvas group visibility controls).
- **Fail Panel:** `failPanel` container, retry, and home navigation buttons.
- **Keep Playing Panel (Revival):** `keepPlayingPanel` container, close button, `playOnButton` (spend coins to continue), `watchAdButton` (watch ad to continue), and `playOnCostText`.
- **Hard Level HUD:** `sliderOut` (normal progress slider frame), `hardLevelSliderOut` (red hard level progress slider frame), and `hardLevelOpen` (visual group indicating hard difficulty).

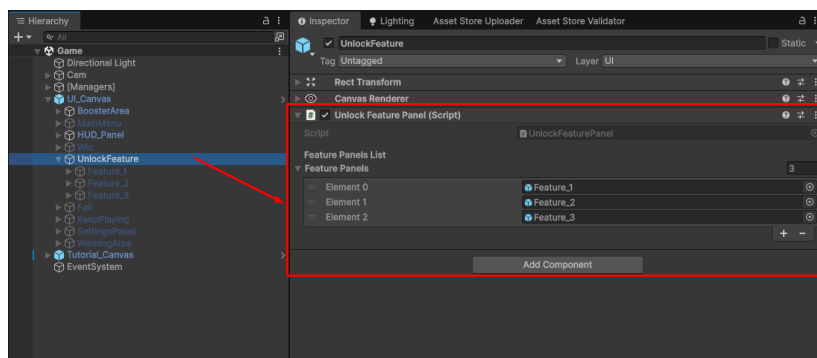
BoosterSlotUI (Booster Button Controller)



Location: `BoosterSlotUI.cs` attached to each individual booster button slot prefab in the HUD.

- **Booster Bindings:** Connects a slot to a specific `BoosterType` (`ExtraSlot` , `SuperShooter` , or `MoveShooter`).
- **UI Elements:** Holds references to the `mainButton` , `lockGroup` (lock group graphic overlay), `lockLvlText` (e.g. "Level 2" unlock warning), `activeGroup` (unlocked state), `deactiveGroup` (locked/inactive state), `countGroup` (shows inventory counts), `countText` , and `plusBuyGroup` (displays "+" purchase icon when inventory is 0).
- **Buy Panel References:** Controls `buyPanel` , coin purchase triggers (`buyWithCoinsButton`), and cost text (`buyPriceText`).
- **Selection Glow (`glowActive`):** Displays a breathing scale animation (`1.0x` to `1.12x` using DOTween loop yoyo) when the player selects `MoveShooter` or `SuperShooter` booster, indicating they need to tap a target block on the grid to execute the booster action.

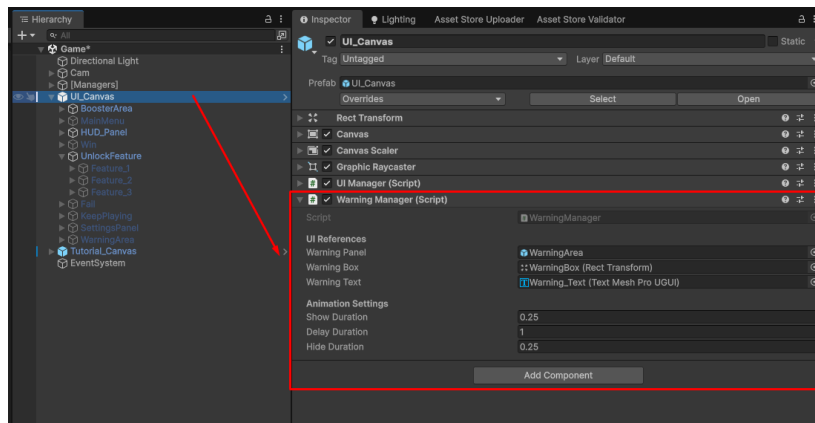
UnlockFeaturePanel (New Feature Introductions)



Location: `UnlockFeaturePanel.cs` attached to the Canvas.

- **Feature Introductions:** Displays tutorial panels introducing new mechanics when players reach specific level thresholds configured in `GameConfig`.
- **Panels List:** Maps game levels to elements inside the `featurePanels` list:
 - `[0] Mystery Shooter` (unlocked at `mysteryShooterUnlockLevel`).
 - `[1] Freeze Shooter` (unlocked at `freezeShooterUnlockLevel`).
 - `[2] Tunnel` (unlocked at `tunnelUnlockLevel`).
- **Lifecycle:** When a level is cleared, `UIManager` calls `TryShowUnlockPanelForLevel()`. If a threshold matches, the corresponding tutorial slides in with DOTween easing. Clicking "Continue" closes the panel and loads the next level.

WarningManager & WarningArea (Popup Alerts)



Location: `WarningManager.cs` attached to the Canvas.

Provides localized, temporary warning popups to display alerts like "Not enough coins!" or "No ads available right now!".

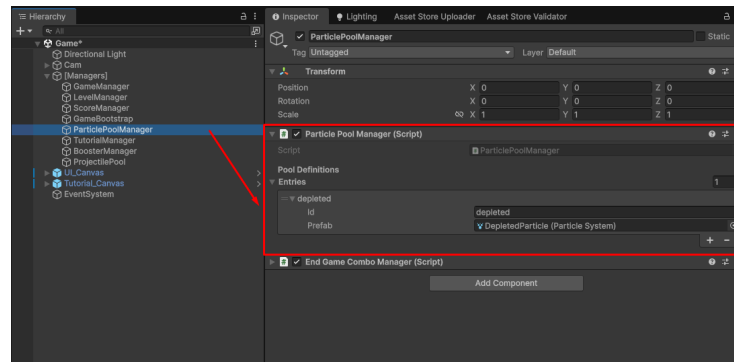
```
#Example Use
if (WarningManager.Instance != null)
{
    WarningManager.Instance.ShowWarning("Not enough coins!");
}
```

- **Structure:** Operates as a Singleton. Manages `warningPanel` (must have a `CanvasGroup` component for alpha fading), `warningBox` (RectTransform for scaling animations), and `warningText` (displays TMPPro warning string).
- **Workflow:** When `ShowWarning("message")` is called:
 1. Existing warning sequences are aborted and killed to prevent overlapping visual bugs.
 2. The panel alpha is reset to `0`, and scale is set to `0`. The panel is set to active.

3. A DOTween Sequence handles fade-in (DOFade to 1) and zoom-in (DOScale to 1 with `OutBack` easing) over `showDuration` (0.25s).
4. Waits for `delayDuration` (1.5s).
5. Runs fade-out (DOFade to 0) and zoom-out (DOScale to 0 with `InBack` easing) over `hideDuration` (0.25s), then disables the panel.
6. Uses `SetUpdate(true)` to guarantee warnings display smoothly even when the game is paused (`Time.timeScale = 0`).

▼ Visual Effects & Game Managers

1. ParticlePoolManager (Pool Manager)

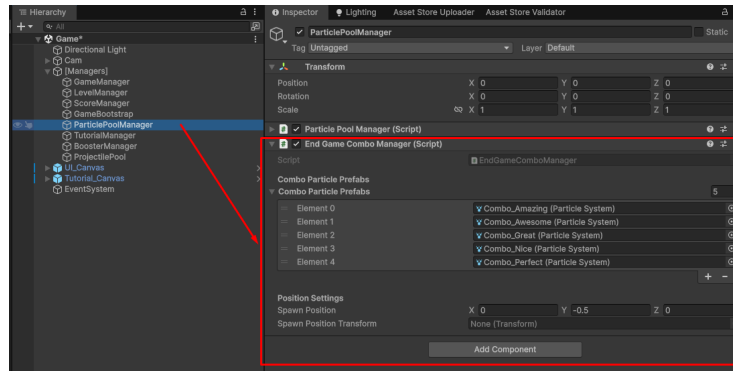


Location: `ParticlePoolManager.cs` attached to the `ParticlePoolManager` GameObject in the Game scene.

Handles world-space visual effects using a performance-optimized object pooling system:

- **Fire-and-Forget Effects:** Plays particles (like box destruction bursts or block depletion effects) using a pool so that they are never cut off when the spawning GameObject is set to inactive (`SetActive(false)`).
- **Pre-Warming System:** Instantiates `3` (DefaultPreWarm) copies of each particle system during initialization. This prevents CPU and Garbage Collection (GC) spikes on the first frame a particle is spawned.
- **Dynamic Expansion:** If all pre-warmed particle instances are busy playing, the pool automatically instantiates new items on the fly to prevent visual skips.
- **Auto-Return coroutine:** Tracks the particle lifecycle. As soon as the emission ends and the particle is dead (`IsAlive == false`), it is automatically deactivated and returned to the queue.

2. EndGameComboManager



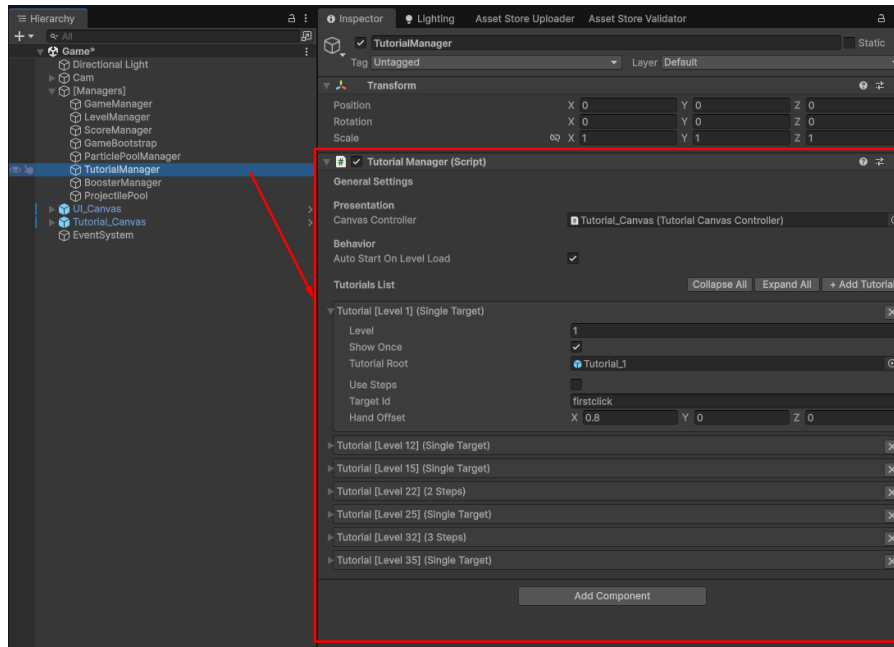
Location: `EndGameComboManager.cs` attached to the `ComboGroupPanel` or manager GameObject.

Triggers special 3D combo particle celebrations during the final stage of gameplay:

- **End Game Phase Detection:** Checks `IsEndGamePhase()` by scanning the shooter grid. The combo stage triggers only when there are no active `InGrid` shooters left and players are firing their remaining ammo directly from the slots.
- **Event Subscription:** Listens to `ShooterBlock.OnAnyBlockDepleted`. Every time a shooter block in an active slot runs out of ammo and is destroyed during the end game phase, a random celebratory combo effect is triggered.
- **Random Combo Selection:** Picks a random effect from the inspector's `comboParticlePrefabs` list (avoiding repeating the last played effect).
- **Pool Integration:** Spawns particles at a custom target position (configured via `spawnPositionTransform`) by routing calls through the `ParticlePoolManager` to ensure smooth rendering.

▼ Tutorial Manager

Location: `TutorialManager.cs` attached to the UI Canvas in the Game Scene.



The tutorial manager is a step-by-step tutorial runner that guides players through new mechanics (like introducing new blocks or boosters) by highlighting elements and blocking unauthorized clicks.

1) How It Works & Level Load Triggers

- **Auto-Start:** If `_autoStartOnLevelLoad` is active, the manager automatically executes on level start.
- **Level Check:** It searches the `_tutorials` configuration list for a definition matching the current level index (`SaveManager.CurrentLevel`).
- **One-Time Execution:** If a tutorial is configured with `ShowOnce` set to true, its completion status is saved via PlayerPrefs (e.g. `TutorialCompleted_Level_X`). It will never repeat on subsequent playthroughs once completed.

2) Adding Tutorials via the Tutorials List

To configure a new tutorial:

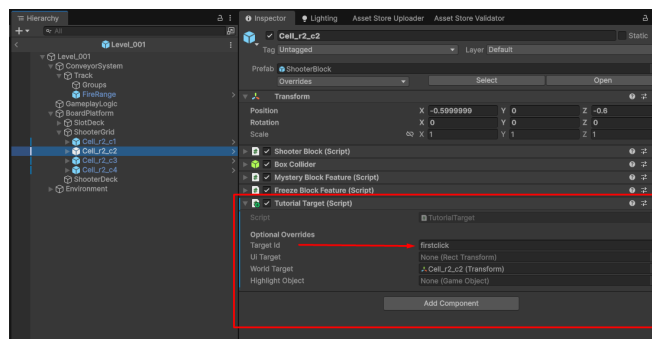
1. Locate the TutorialManager in the Game Scene.
2. Select the `TutorialManager` component.
3. Add a new element to the **Tutorials** (`_tutorials`) list:
 - **Level:** The level index where this tutorial will trigger.
 - **ShowOnce:** If checked, this tutorial will only play once per player account.

- **Tutorial Root:** Assign the parent UI GameObject containing the step windows or dialog cards.
- **Use Steps:** Check this if you want multiple sequential step actions. If unchecked, it behaves as a single-step highlight tutorial.
- **Target Id:** (Used for single-step tutorials) The unique string ID matching the target element.
- **Hand Offset:** Local offset coordinate for the tutorial hand indicator.
- **Steps:** (Used if `UseSteps` is enabled) An ordered list of steps described below.

3) Target IDs & Target Tutorials

The system uses string ID matching to dynamically locate screen-space or world-space objects without hardcoding scene dependencies:

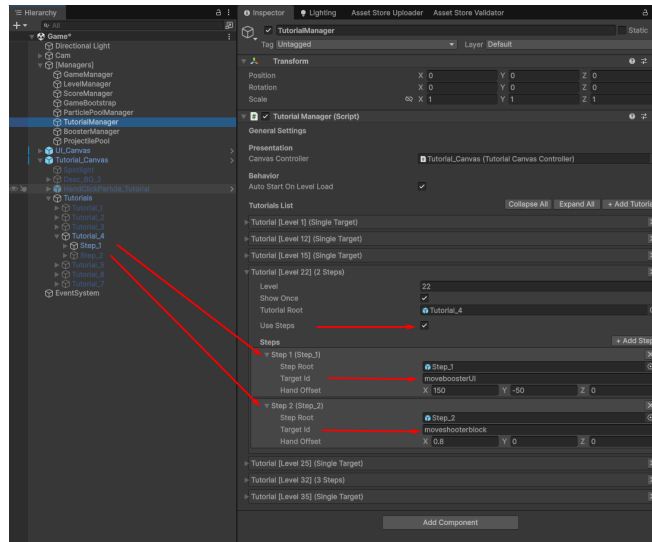
- **TutorialTarget Component:** Any object (buttons, grid cells, active slots, booster panels) that needs to highlight during a tutorial must have a `TutorialTarget` component attached with a unique **Target ID** specified.



- **Target Hand Tracking:** When a tutorial step starts, the manager queries registered targets for the current step's `TargetId`. If found, the `TutorialCanvasController` moves the pointing hand icon directly over the target's position (supporting both UI transforms and world targets).
- **Input Blocking:** The manager intercepts all player inputs using `TryHandleTargetClick`.
 - If the player clicks the highlighted target (matching the active ID), the click passes through to gameplay, and the tutorial advances or completes.
 - If the player clicks anywhere else, the click is blocked, shielding the gameplay from wrong actions.

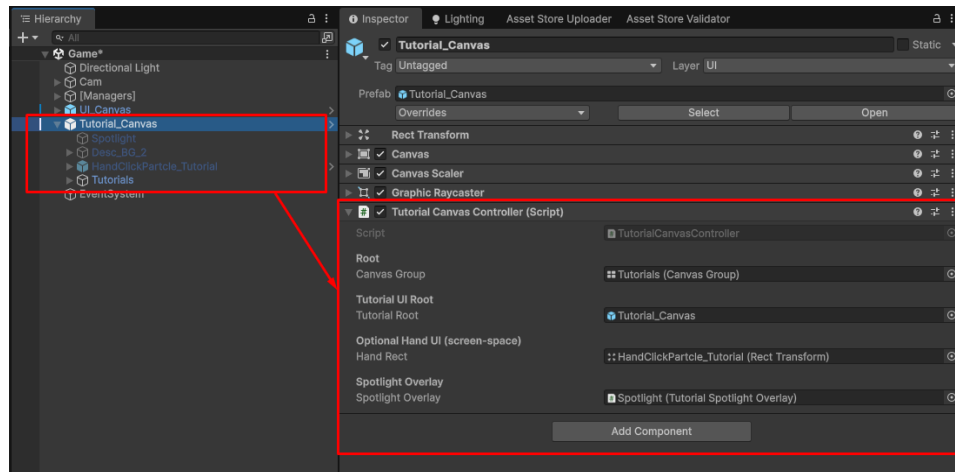
4) Step-by-Step Tutorials

Step-by-step tutorials allow you to guide a player through a series of logical inputs (e.g. "Tap Red Shooter" → "Drag to Slot" → "Watch projectile hit conveyor").



- **Configuration:** Inside a tutorial definition, check **Use Steps** and populate the **Steps** array.
- **TutorialStepDefinition Fields:**
 - **Step Root:** The UI GameObject representing the text dialog or visual elements for this specific step.
 - **Target Id:** The ID of the **TutorialTarget** that the player must tap to advance.
 - **Hand Offset:** Custom position adjustments for the pointing hand.
- **Workflow:**
 1. The manager sets the first step's **StepRoot** to active and searches for the target ID.
 2. The pointer hand points at the target.
 3. Once the user clicks the target, the active **StepRoot** is set to inactive.
 4. The index increments, activating the next step's root and target ID.
 5. The loop finishes when all steps in the array are completed.

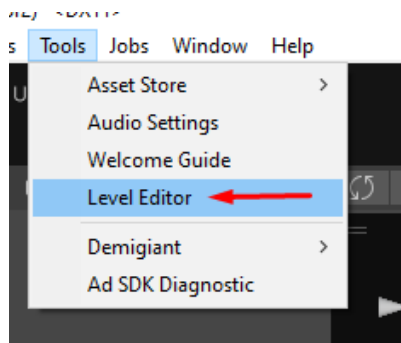
5) Visual References: Tutorial Canvas Setup



You can inspect existing templates inside the **Tutorial Canvas** hierarchy to see how they are structured:

- **Spotlight Mask (`TutorialSpotlightOverlay`)**: Uses a canvas mask shader to darken the entire screen except for a round transparent circle focusing on the target.
- **Hand Pointer (`_handRect`)**: A UI image of a hand indicator that updates its anchored position dynamically in `LateUpdate` to float directly above UI targets or screen-projected world coordinates.
- **Step Dialog Prefabs**: Built using standard TMPPro text cards. Make copies of existing step prefabs in the UI Canvas under Tutorial targets to design custom text boxes.

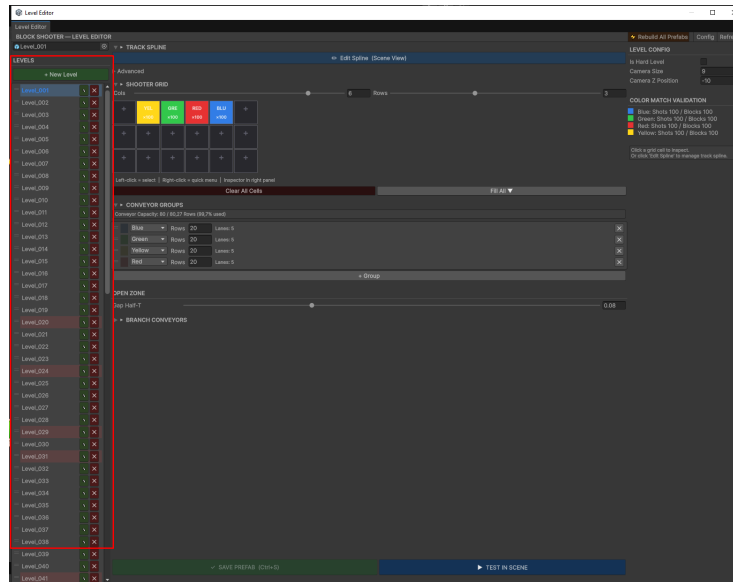
▼ Using the Custom Level Editor Panel



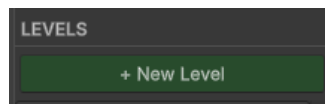
Access it via: `Tools -> Level Editor`

The level editor is a comprehensive Unity Editor Window tool designed to build level prefabs without changing scenes.

1) Level List & Configuration Actions



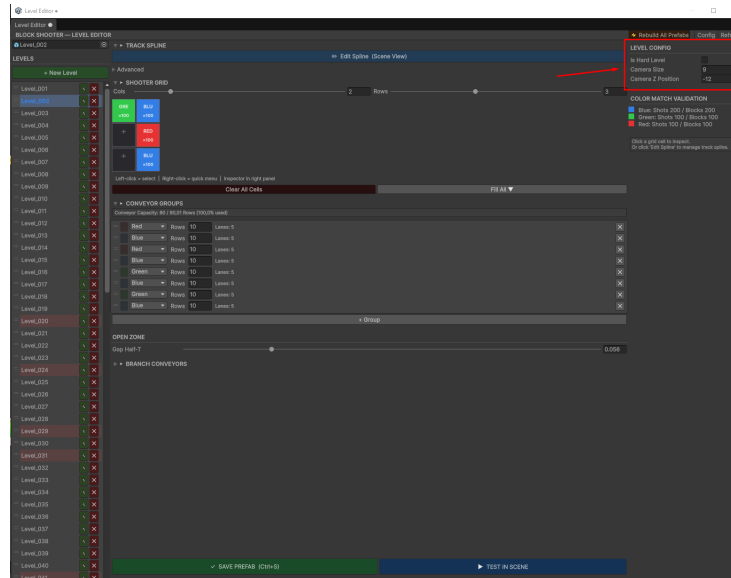
- **New Level:** Instantiates a clean level prefab in the editor scene, auto-assigned with the next sequence number.



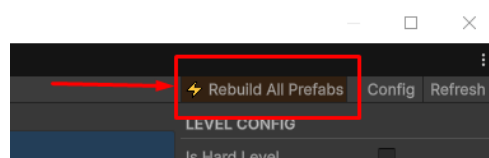
- **Level Duplication:** Easily duplicate an existing level to copy its settings, grid deck, and path details.



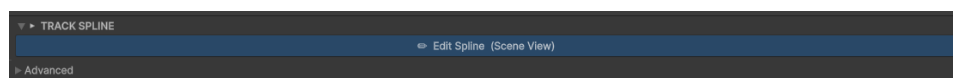
- **Level Deletion:** Delete levels safely, which automatically triggers index adjustments.
- **Level Settings Config:**



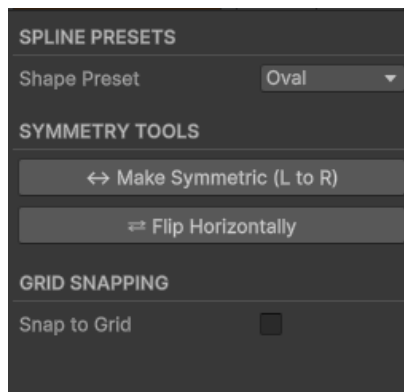
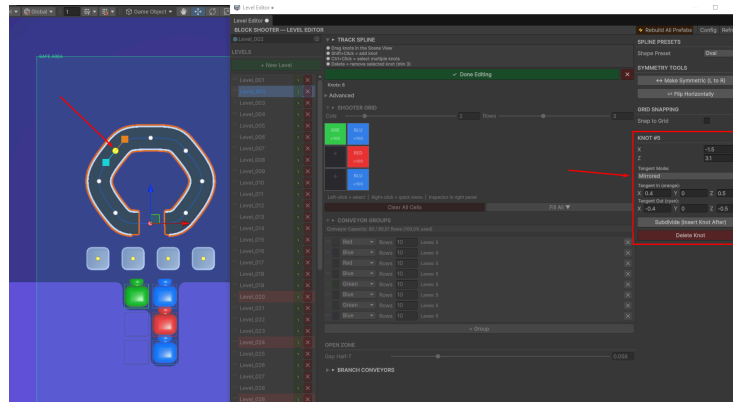
- **Hard Level** : Toggle this to mark the level as hard. Hard levels display with a **red frame** in the level selection Menu road.
- **Camera Size** & **Camera Z Position** : Tweak custom camera metrics specifically for this level.
- **Rebuild All Prefabs**: Click this to regenerate and save all level prefabs across the project in one batch action.



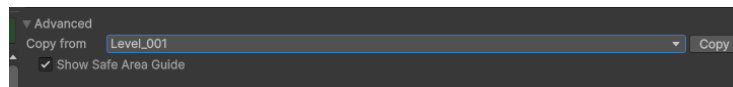
2) Spline Editor (Conveyor Tracks)



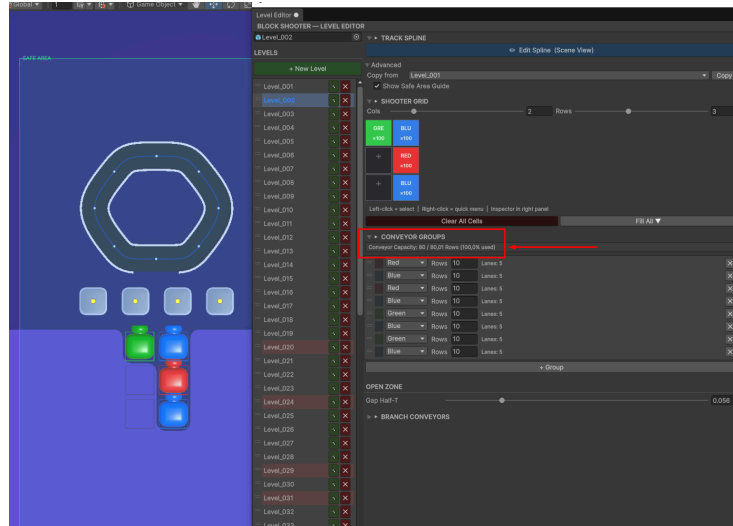
Create and manipulate conveyor tracks directly in the Scene view:



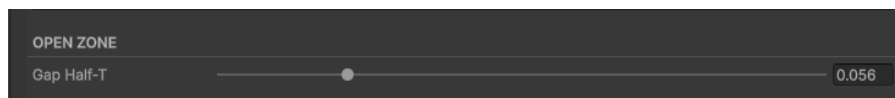
- **Edit Spline Mode:** Drag knots directly in the scene to sculpt the conveyor path.
- **Undo/Redo Support:** Supports **Ctrl+Z** (Undo) and **Ctrl+Y** (Redo) history tracking during spline editing.
- **Symmetry Tools:** Duplicate paths symmetrically to design mirrored layouts.
- **Grid Snapping:** Turn on snapping to align path knots with grid units.
- **Copy From Spline:** Click **Copy From** to duplicate the path spline layout from another level instantly.



- **Conveyor Capacity Tool:** Displays real-time calculations showing exactly how many color block rows fit on the current spline track length. Use this index to balance your conveyor contents.

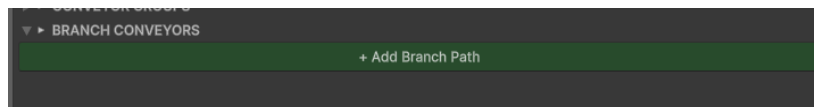


- **Open Zone Gap Settings:** Modify segment spacing parameters (**gap**) to customize open spaces along the track.

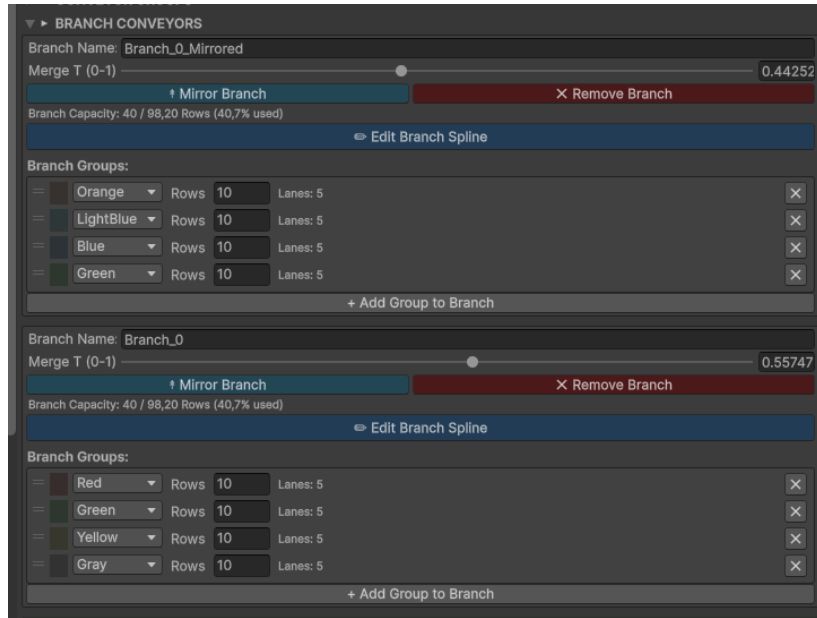


3) Branch Conveyors (Sub-Tracks)

- **Add Branch Splines:** Add supplementary tracks that merge into the main conveyor track.



- **Edit Branch Spline:** Build secondary tracks in the Scene view. The branch path endpoint automatically targets and links to the main conveyor.

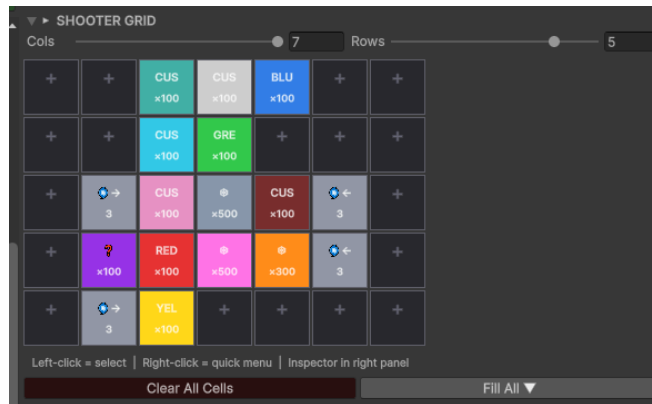


- **Mirror Branch:** Click **Mirror Branch** to duplicate and mirror your active branch path.
- **Add Group to Branch:** Populate branches with block color groups. Configure the **rows** parameter. Note that the branch lane count is fixed at **5**.

4) Conveyor Groups & Colors

- Select a conveyor group to add color box sequences. Assign color orders to spawn down the tracks.

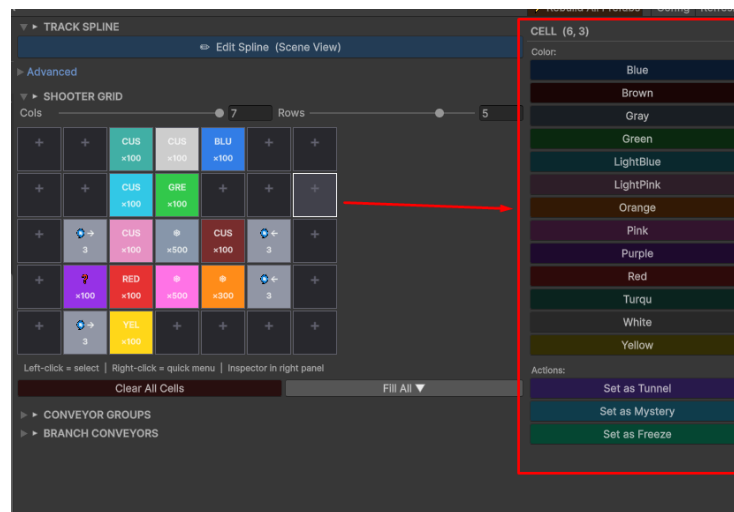
5) Shooter Grid Editor (Deck Layout)



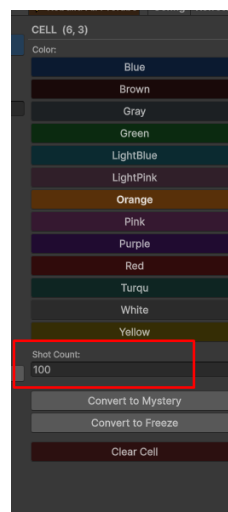
Design the playable deck layout using a 7×6 cell matrix:

- **Grid Keyboard Navigation:** Navigate through cells in the editor panel using **WASD** or **Arrow keys**.

- **Cell Color Selection:** Click a grid cell to assign block colors from the registry in the right panel.



- **Shot Count:** Set customized ammo limits for each shooter block.



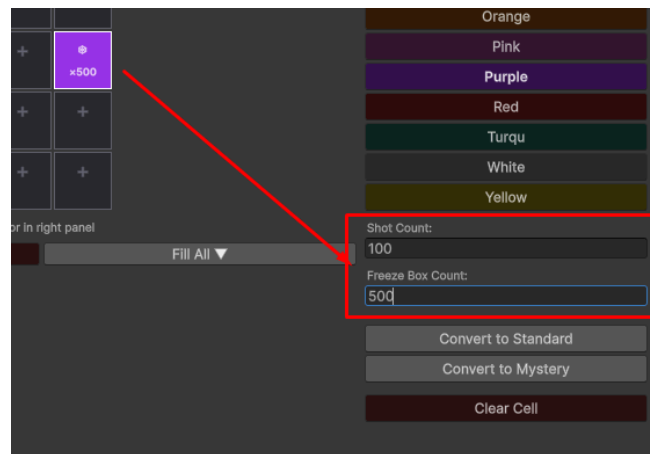
- **Clear Cell / Fill All:** Wipe individual cell configurations or fill the entire grid layout instantly.
- **Clear All Cells:** Completely clears the active deck grid.

6) Level-Locked Features (Progression Rules)

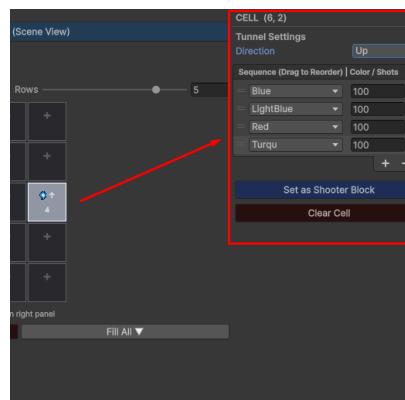
To ensure a balanced player learning curve, advanced elements enforce unlock level thresholds:

- **Unlock Threshold Rule:** Elements like Mystery blocks, Frozen blocks, and Tunnels are hidden and unavailable in the cell editor if the active level is below their unlock settings (configured in `GameplaySettingsConfig`).

- *Example:* If Mystery Blocks unlock at level 15, clicking a cell in level 10 will not show Mystery options under Actions. They only unlock for levels 15 and higher.
- **Mystery Blocks:** Reveal their color only when loaded into slot.
- **Frozen Blocks:** Enable these by selecting a cell and setting the `FreezeBoxCount` value. Players must hit adjacent blocks to break the ice cover before these shooters become active.



- **Tunnels:** Place tunnels along the spline and configure the `Direction` and `Sequence` (block colors and shot count details exiting the tunnel) under the Tunnel Settings panel.



7) Saving, Testing & Validation



- **Prefab Serializing:** Save your work by pressing `Ctrl+S` or clicking **Save Prefab**.
- **Test Scene Button:** Click **Test Scene** to jump directly into the level in play mode for instant feedback.

- **Color Match Validation:** Checks if the block colors spawned on the conveyor tracks match the available shooter colors on the deck grid to prevent unwinnable level designs.

▼ Level Generation & Models Directory

- **Prefabs Directory:** Level prefabs are generated and saved under `Assets/Project Files/Data/Levels/`.
- **Models Directory:** All generated spline track meshes and deck models are saved under `Assets/Project Files/Game/Models/LevelMesh/`.

▼ Core Systems Configuration

Ads Settings Configuration

- **Configuration Asset:** `AdsSettings.asset` under resources.
- **Supported Ads:** Banner ads, Interstitial ads, and Rewarded video ads.
- Supports GDPR consent and multiple ad network providers (AdMob, Unity Ads, AppLovin LevelPlay).

IAP Settings Configuration

- **Configuration Asset:** `IAPSettings.asset` under resources.
- **Supported Products:** Coin packages, Booster packages, and No Ads removal.
- Connects directly to App Store / Google Play store console IDs.

Audio System Configuration

- **Configuration Asset:** `AudioData.asset` under resources.
- Manages background music tracks, sound effects (SFX) clips, and default volume controls for Master, Music, and SFX channels.

Save System

- **Static Data Handler:** Handled by the static `SaveManager` class.
- **Storage Technology:** Utilizes Unity's native `PlayerPrefs` for local device saving, ensuring cross-platform compatibility out of the box.
- **Auto-Save Functionality:** Automatically triggers `PlayerPrefs.Save()` immediately after modifying coins, levels, or booster counts to prevent progress loss.
- **Saved Parameters & Registry Keys:**
 - **Current Level** (`"CurrentLevel"`): Tracks the player's active stage progress (default starts at `1`).

- **Coins** (`"Coins"`): Tracks the player's coin balance. It automatically initializes using the configured `GameConfig.startGameCoins` value (fallback is `100`).
- **Booster Inventory**: Tracks counts for each usable booster type:
 - **Extra Slot** (`"Booster_ExtraSlot"`)
 - **Super Shooter / Color Blast** (`"Booster_ColorBlast"`)
 - **Move Shooter** (`"Booster_MoveShooter"`)
- **Reset Utility**: Includes a `ClearAll()` method which runs `PlayerPrefs.DeleteAll()` to clear player history, which is linked directly to the developer QA testing panel under GameConfig.

▼ Build Settings & Troubleshooting

Before Building Checklist

1. Scenes in Build:

- `Loading` scene at index 0.
- `Menu` scene at index 1.
- `Game` scene at index 2.

2. Configuration Check:

- Verify all levels are added to the `LevelSequenceConfig.asset` .
- Ensure IAP identifiers match Apple/Google consoles.
- Check that the `AdsSettings` test mode is disabled for production releases.

3. Data Reset:

- Reset local storage files before building to ensure a clean first-install experience for players.

Troubleshooting

- **Pink/Missing Materials**: Verify your target platform supports standard shader compilation. Check that all materials inside `ColorRegistryConfig.asset` are correctly configured.
- **Haptics Not Triggering**: Ensure haptic feedback is toggled on in the Menu settings, and the `Vibration` package is correctly imported.
- **Spline Mesh Deformation**: If track rails look jagged, increase the `trackResolution` inside `LevelEditorConfig.asset` .